

Computer Architecture Mini-Project 3 Report

Introduction

This report details the design and implementation of a digital circuit using the OSS CAD suite to generate a sinusoidal waveform through a 10-bit R-2R ladder DAC. The circuit is optimized to use minimal memory storage by leveraging symmetry within one cycle of the sine function. The FPGA development board used is the iceBlinkPico, which features an iCE40UP5K FPGA from Lattice Semiconductor. The project was implemented in SystemVerilog, tested using Icarus Verilog (iverilog), and measured using an Analog Discovery 2 oscilloscope. This report describes the circuit design, operation, simulation results, and provides the source code used.

Circuit Design

The circuit is composed of two main modules described below:

1. **memory.sv:** implements a quarter-cycle sine wave lookup table for generating sinusoidal wave values.
 - Stores pre-computed 9-bit amplitude values for a quarter sine wave in a 128-entry memory array, initialized from `INIT_FILE`.
 - Uses a 9-bit input signal, `read_address`, to specify the lookup index for retrieving sine wave values, dividing the full sine cycle into four quadrants.
 - For each quadrant, the amplitude is adjusted based on symmetry:
 - i. Q1 ($0 - \pi/2$): Directly reads the stored amplitude.
 - ii. Q2 ($\pi/2 - \pi$): Reads a mirrored version of the stored amplitude.
 - iii. Q3 ($\pi - 3\pi/2$): Reads the stored amplitude and negates it.
 - iv. Q4 ($3\pi/2 - 2\pi$): Reads a mirrored version of the stored amplitude and negates it.
 - Outputs a 10-bit sine wave value (`read_data`) centered around 512 to represent a full sine wave cycle.
2. **sine.sv:** generates a 10-bit sine wave output by sequentially reading pre-computed values from memory.
 - Implement a 9-bit address counter that increments on each clock cycle to sequentially read sine wave values.
 - Instantiates the `memory` module, which stores a quarter-cycle sine wave lookup table initialized from `"sine.txt"`.
 - Uses the `read_address` signal to retrieve 10-bit sine values from memory, cycling through a full sine wave.
 - Assigns the 10-bit output `data` to individual output signals (`_48b` to `_9b`), mapping each bit to a corresponding output pin.

Circuit Operation

The circuit operates as follows:

1. The system clock (`clk`) drives the `top` module, incrementing the `address` counter on each clock cycle.
2. The `address` signal is fed into the `memory` module, which retrieves the corresponding 10-bit sine wave value from a preloaded lookup table (`sine.txt`).
3. The retrieved sine wave value (`data`) is output as a 10-bit signal.
4. Each bit of the `data` output is mapped to individual output signals (`_48b` to `_9b`), representing different digital outputs.
5. The address counter continuously increments, cycling through the full sine wave, repeating indefinitely.

Simulation Results

To validate the design, a SystemVerilog testbench (`sine_tb.sv`) was created to simulate the circuit using Icarus Verilog. The testbench functions as follows:

- Generates a clock signal with a period of 8ns, toggling every 4ns.
- Instantiates the `top` module and connects its output signals (`_9b` to `_48b`).
- Monitors and prints the values of the sine wave output signals at each time step for debugging.
- Captures simulation results in a VCD (value change dump) file (`sine.vcd`) for waveform visualization in GTKWave.

A screenshot of the waveform is provided below. It shows the `read_data` signal forming a smooth sinusoidal wave. This confirms the expected sine wave generation from the lookup table, demonstrating the memory-based waveform generation.

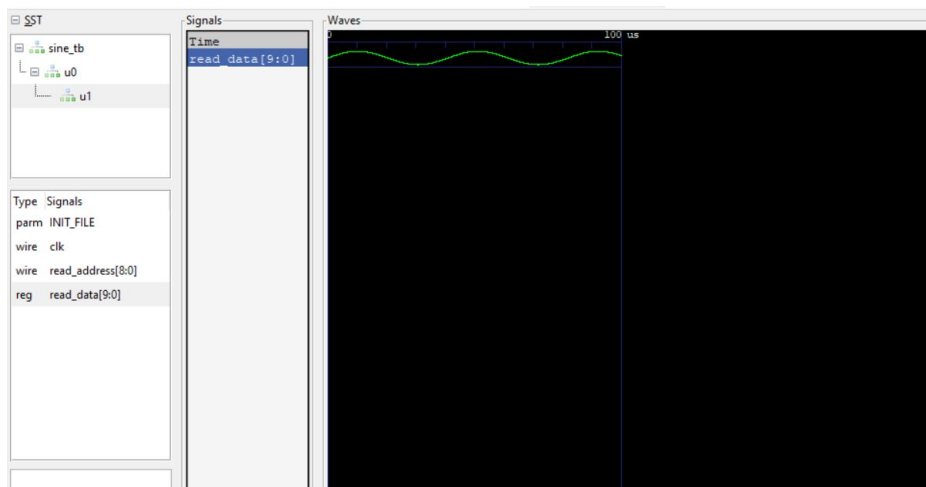


Figure 1: Simulated sine wave output from the lookup table in GTKWave.

Oscilloscope Measurement

The sine wave output was measured using the Analog Discovery 2 oscilloscope. The captured waveform, shown in Figure 2, confirms the expected sinusoidal signal shape. The oscilloscope was set to a 50 MHz sampling rate with a 10 $\mu\text{s}/\text{div}$ time base. The signal maintains smooth transitions, validating the correct operation of the sine wave generator.

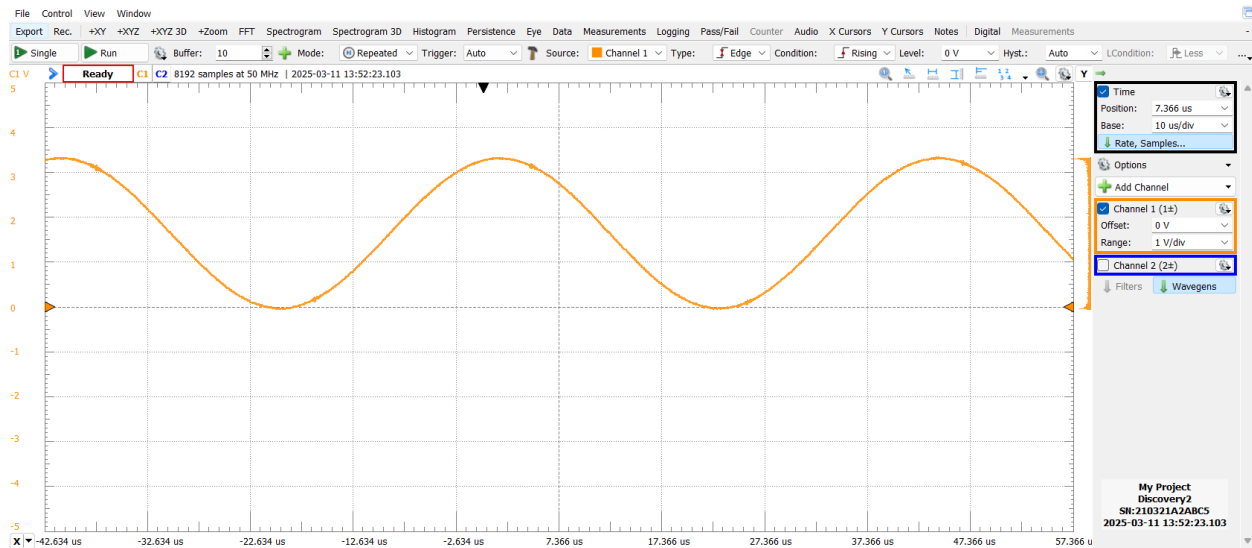


Figure 2: Oscilloscope capture of the generated sine wave signal.

Source Code

Sine_tb.sv

```
`timescale 10ns/10ns
`include "sine.sv"

module sine_tb;

    logic clk = 0;
    logic _9b, _6a, _4a, _2a, _0a, _5a, _3b, _49a, _45a, _48b;

    top u0 (
        .clk      (clk),
        ._9b      (_9b),
        ._6a      (_6a),
        ._4a      (_4a),
        ._2a      (_2a),
        ._0a      (_0a),
        ._5a      (_5a),
        ._3b      (_3b),
        ._49a     (_49a),
        ._45a     (_45a),
        ._48b     (_48b)
    );

    initial begin
        $dumpfile("sine.vcd");
        $dumpvars(0, sine_tb);

        // Debugging: Monitor values every cycle
        $monitor("Time: %0t | _9b: %b _6a: %b _4a: %b _2a: %b _0a: %b _5a: %b _3b: %b _49a: %b _45a: %b _48b: %b",
            |   |   | $time, _9b, _6a, _4a, _2a, _0a, _5a, _3b, _49a, _45a, _48b);

        #10000
        $finish;
    end

    always begin
        #4
        clk = ~clk;
    end

endmodule
```

Sine.sv

```
`include "memory.sv"

// Sine Wave Top-Level Module

module top(
    input logic    clk,
    output logic   _9b,    // D0
    output logic   _6a,    // D1
    output logic   _4a,    // D2
    output logic   _2a,    // D3
    output logic   _0a,    // D4
    output logic   _5a,    // D5
    output logic   _3b,    // D6
    output logic   _49a,   // D7
    output logic   _45a,   // D8
    output logic   _48b    // D9
);

    logic [8:0] address = 0;
    logic [9:0] data;

    memory #(
        .INIT_FILE      ("sine.txt") // Uses only 128 samples
    ) u1 (
        .clk             (clk),
        .read_address    (address),
        .read_data       (data)
    );

    always_ff @(posedge clk) begin
        address <= address + 1;
    end

    assign {_48b, _45a, _49a, _3b, _5a, _0a, _2a, _4a, _6a, _9b} = data;
endmodule
```

Memory.sv

```
module memory #(
    parameter INIT_FILE = ""
)(
    input logic clk,
    input logic [8:0] read_address,
    output logic [9:0] read_data
);

    logic [8:0] quarter_cycle_memory [0:127]; // 9-bit amplitude (0-511)

    initial if (INIT_FILE) begin
        $readmemh(INIT_FILE, quarter_cycle_memory);
    end

    always_ff @(posedge clk) begin
        case (read_address[8:7])
            // Q1: 0- $\pi/2$  -> 512 to 1023 (512 + amplitude)
            2'b00: read_data <= 10'd512 + {1'b0, quarter_cycle_memory[read_address[6:0]]};

            // Q2:  $\pi/2$ - $\pi$  -> 1023 to 512 (512 + mirrored amplitude)
            2'b01: read_data <= 10'd512 + {1'b0, quarter_cycle_memory[127 - read_address[6:0]]};

            // Q3:  $\pi$ - $3\pi/2$  -> 512 to 1 (512 - amplitude)
            2'b10: read_data <= 10'd512 - {1'b0, quarter_cycle_memory[read_address[6:0]]};

            // Q4:  $3\pi/2$ - $2\pi$  -> 1 to 512 (512 - mirrored amplitude)
            2'b11: read_data <= 10'd512 - {1'b0, quarter_cycle_memory[127 - read_address[6:0]]};
        endcase
    end
endmodule
```